Lesson plan · for the teacher

Counts and averages: GROUP BY

Data engineering · Level 3 · Lesson 3

Time	Materials
40 min	Board and chalk; Space for pupils to stand in groups; Optional: a phone or computer for the SQL steps

Learning goals

- Summarise with COUNT, AVG, MIN and MAX.
- Make one row per group with GROUP BY, and filter groups with HAVING.
- Handle NULL, and check summaries for glitches.

Activity: Grouping by hand

1. Warm-up (5 min): Ask how long it takes to work out every class's average by hand, and how often the school does it.
2. Teach (8 min): Show COUNT, AVG, MIN and MAX on a small table, then the same with GROUP BY class. Explain WHERE before and HAVING after, and what NULL does.
3. Group (12 min): Pupils stand in groups by birth month. Each group counts itself and works out its average travel time; the class writes one row per group.
4. Filter (10 min): Cross out the groups with three pupils or fewer: that's HAVING COUNT(*) > 3. Then ask for each group's lowest and highest: any glitches?
5. Wrap-up (5 min): Write the query for what the class just did. Then give the exit ticket.

Pupils may choose a month instead of giving their real birth month. Keep groups small enough to count quickly, and have one pupil whose travel time is "not sure" to show what NULL does.

Discussion

- Why can't we select each pupil's name in a GROUP BY class query?
- What's the difference between COUNT(*) and COUNT(mark)?
- Why look at MIN and MAX before an average?

Exit ticket

- Write a query for the number of pupils in each class, keeping only classes with more than 30.

Try it at school tomorrow

Pupils group themselves by birth month. Each group counts itself, and the class writes one row per month on the board: that's GROUP BY, done with people.

Practice questions · for you

Counts and averages: GROUP BY

Data engineering · Level 3 · Lesson 3

1 **After GROUP BY class, which line keeps only classes with more than 30 pupils?**

◯ ORDER BY 30 ◯ WHERE COUNT(*) > 30 ◯ HAVING COUNT(*) > 30

2 **The district wants each class's average maths mark. Show the class and the average rounded to 1 decimal place, one row per class.**

```
SELECT class, AVG(mark) FROM marks;
```

Write your program:

3 **The school has 9 science rows; Musa's mark is NULL, and the other 8 add up to 106. What does this query return?**

```
SELECT COUNT(*), COUNT(mark), AVG(mark)
FROM marks
WHERE subject = 'science';
```

◯ 9, 8, 13.25 ◯ 8, 8, 13.25 ◯ 9, 9, 11.78

4 **The absences table has one row for every absence this week. Which classes had more than 3 absences? Show the class and its number of absences.**

```
SELECT class, COUNT(*) FROM absences
GROUP BY class;
```

Write your program:

5 **Match each summary to what it tells you.**

COUNT(*)	• • • •	The average of the marks that are there
COUNT(mark)	• • • •	How many rows, empty marks included
AVG(mark)	• • • •	The lowest and highest: look here for glitches
MIN and MAX	• • • •	How many rows have a mark

✎ **Which summary does your school work out by hand every term? Write the GROUP BY query that would do it.**

Check your answers

Counts and averages: GROUP BY

Data engineering · Level 3 · Lesson 3

1 **HAVING COUNT(*) > 30**

HAVING filters groups after they're counted.

2

```
SELECT class, ROUND(AVG(mark), 1) FROM marks
WHERE subject = 'maths'
GROUP BY class;
```

WHERE keeps the maths rows, GROUP BY class makes one group per class, and ROUND(AVG(mark), 1) summarises each: 5A 11.0, 6A 10.0, 6B 12.0.

3 **9, 8, 13.25**

COUNT(*) counts all 9 rows; COUNT(mark) skips the NULL, so 8; and AVG(mark) divides 106 by the 8 real marks: 13.25.

4

```
SELECT class, COUNT(*) FROM absences
GROUP BY class
HAVING COUNT(*) > 3;
```

GROUP BY class counts each class's absences: 6A 5, 6B 2 and 5A 4.

5 **COUNT(*) — How many rows, empty marks included**
COUNT(mark) — How many rows have a mark
AVG(mark) — The average of the marks that are there
MIN and MAX — The lowest and highest: look here for glitches

COUNT(*) and COUNT(mark) differ exactly when marks are missing, and AVG skips those.